

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_b9d6fa25-e41\agent-acb37af8aff41ad77.jsonl`  
- SHA-256: `ee2f19f45f4313a29bf5004321122993d244b643eca6de1be9f0dcb22e59d426`  
- Source modified: `2026-06-25T17:38:27+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_b9d6fa25-e41`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:35:10.740Z)

PR #385 "fix(B1/P9): remove fabricated data from motion segmenter" is CHECKED OUT (branch b1-p9/remove-fabricated-motion-data). The motion segmenter (the LIGHT-tier DEFAULT segmenter) used to FABRICATE rich metadata with the stdlib random module ? server/receiver (random.sample), shot_count (random.randint), avg_speed_kmh (random.uniform), and serve/smash/foul/winner EVENTS ? persisted as if real. This PR removes the fabrication: passes None for the un-modeled fields, DROPS all db.add_event calls, removes the random import. It KEEPS the real signal (segment start/end/duration from pixel-diff detection; intensity derived from detected duration). Changed files (read the real diff: git diff origin/master...HEAD):
backend/pipeline/segmenters/motion.py, tests/test_motion_segmenter.py. CI is GREEN ? do NOT re-run CI; review the CODE.

*** STRICT READ-ONLY: this is a REVIEW. You MUST NOT call Edit/Write/NotebookEdit or modify ANY file. Only Read/Grep/Glob/Bash-to-inspect. If you change a file the review is INVALID. ***

Review ONLY the downstream BLAST RADIUS ? motion is the LIGHT-tier DEFAULT segmenter, so its output is what most local users get. THIS IS THE HIGHEST-RISK DIMENSION ? grep the whole codebase. (a) Who READS the now-None fields (server/receiver/shot_count/avg_speed) ? the stitcher/compiler, reporting/harvest, reel watermark/metadata, the API/JSON response, the UI/SPA? Does any consumer assume a non-None number and break (e.g. format/round/compare on None)? (b) The DROPPED events: confirm motion was truly the ONLY db.add_event caller (grep), and find anything that READS events (a report, export, the UI) that would now be empty for motion runs. (c) Any DB-schema/read concern from None / missing events. Report concrete findings with the CONSUMER file:line + the breakage + a fix.

2. assistant (2026-06-25T17:35:13.143Z)

I'll review the downstream blast radius of this motion segmenter change. Let me start by reading the actual diff and understanding what fields are now None.

3. user (2026-06-25T17:35:14.342Z)

```
diff --git a/backend/pipeline/segmenters/motion.py b/backend/pipeline/segmenters/motion.py  
index 9b714c5..36d6b36 100644  
--- a/backend/pipeline/segmenters/motion.py  
+++ b/backend/pipeline/segmenters/motion.py  
@@ -1,5 +1,4 @@  
     import logging  
-import random  
     import time  
     from typing import Any, Dict, List, Optional, Tuple  
  
@@ -22,12 +21,11 @@ class MotionPlaySegmenter(BasePlaySegmenter):  
  
     def process_video(self, video_id: str, video_path: str, player_pool: Optional[List[str]] =
```

```

None, max_frames: Optional[int] = None) -> "Result[List[Dict[str, Any]]]":
    """
-     Processes a video to detect play segments using motion heuristics,
-     populates SQLite with detected segments, and indexes detailed events.
+     Processes a video to detect play segments using motion heuristics and
+     populates SQLite with detected segments. The ``player_pool`` arg is
+     accepted for ABC signature compatibility but no longer used (the motion
+     segmenter has no ground truth for server/receiver ? see _populate_db).
    """
-     if not player_pool:
-         player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
-
-     cap = cv2.VideoCapture(video_path)
-     if not cap.isOpened():
-         logger.error(f"Segmenter could not open video: {video_path}")
@@ -35,7 +33,7 @@ class MotionPlaySegmenter(BasePlaySegmenter):

    final_segments = self._detect_motion_segments(cap, max_frames)

-     segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)
+     segments_data = self._populate_db(video_id, final_segments)

    return Success(value=segments_data)

@@ -184,81 +182,35 @@ class MotionPlaySegmenter(BasePlaySegmenter):

    return final_segments

-     def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
-     player_pool: List[str]) -> List[Dict[str, Any]]:
-     """Persist ``final_segments`` and their fabricated metadata/events to the DB.
+     def _populate_db(self, video_id: str, final_segments: List[Tuple[float, float]]
+     ) -> List[Dict[str, Any]]:
+     """Persist ``final_segments`` to the DB with honest metadata.

-     For each detected segment this assigns a random server/receiver, fabricates
-     rich tags, then indexes a serve / 0?3 smash / termination event set ? all
-     with the stdlib ``random`` module (the legacy demo's fabricated baseline).
-     Returns the per-segment dicts handed back to the caller. The ``random``
-     call order here is load-bearing for reproducibility and must not change.
+     The motion segmenter detects play segments via pixel-diff heuristics but
+     has NO ground truth for who served, shot counts, shuttle speed, or event
+     breakdown ? those were previously fabricated with the stdlib ``random``
+     module (the legacy demo baseline). They are now persisted as ``None`` /
+     unknown markers so consumers can distinguish "not modeled" from a real
+     value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
    """
-     # Populate SQLite DB with segments and metadata events
-     segments_data = []
-     for start, end in final_segments:
-         # Assign Server and Receiver randomly from pool
-         server, receiver = random.sample(player_pool, 2)
-
-         # Formulate dynamic rich tags
-         dur = end - start
-         metadata = {
-             "shot_count": random.randint(4, 25),
+             "shot_count": None,
-             "intensity": "high" if dur > 12 else "medium",
-             "avg_speed_kmh": round(random.uniform(40, 85), 1)
+             "avg_speed_kmh": None,
+         }

-         segment_id = self.db.add_play_segment(video_id, start, end, server, receiver,

```

```

metadata)
-
-     if segment_id:
-         # Add sub-segment events
-         # A: Serve Event (always at start)
-         serve_foul = random.random() < 0.15 # 15% chance of serve foul
-         self.db.add_event(
-             segment_id=segment_id,
-             timestamp=start + 0.5,
-             event_type="serve",
-             player=server,
-             details={"status": "foul" if serve_foul else "valid", "type": "low_serve"
if random.random() > 0.5 else "high_serve"}
-         )
-
-         # B: Smash Events (1 to 3 random smashes per segment)
-         smashes_count = random.randint(0, 3)
-         for _ in range(smashes_count):
-             smash_time = random.uniform(start + 1.5, end - 1.0)
-             smasher = random.choice([server, receiver])
-             self.db.add_event(
-                 segment_id=segment_id,
-                 timestamp=round(smash_time, 2),
-                 event_type="smash",
-                 player=smasher,
-                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
-             )
-
-         # C: Segment Termination Event (foul/winner)
-         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
-         is_foul = random.random() > 0.4
-         error_player = random.choice([server, receiver])
-
-         self.db.add_event(
-             segment_id=segment_id,
-             timestamp=end - 0.2,
-             event_type="foul" if is_foul else "winner",
-             player=error_player,
-             details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
-         )
-
-         # Store segment representation to return to caller
-         segments_data.append({
-             "id": segment_id,
-             "start_time": start,
-             "end_time": end,
-             "duration": dur,
-             "server": server,
-             "receiver": receiver,
-             "metadata": metadata
-         })
-
+         segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
+         segments_data.append({
+             "id": segment_id,
+             "start_time": start,
+             "end_time": end,
+             "duration": dur,
+             "server": None,
+             "receiver": None,
+             "metadata": metadata,
+         })
+     return segments_data

```

```

# Register the segmenter
diff --git a/tests/test_motion_segmenter.py b/tests/test_motion_segmenter.py
index 8d6a710..228531e 100644
--- a/tests/test_motion_segmenter.py
+++ b/tests/test_motion_segmenter.py
@@ -18,7 +18,6 @@ extracted into private helpers without changing observable output:
     fabricated server/receiver/shot_count/events are reproducible. The random call
     ORDER inside process_video is what the seeded assertions pin.
     """
-import random
+from unittest.mock import MagicMock, patch

import backend.pipeline.segmenters.motion as motion_mod
@@ -145,42 +144,36 @@ def _high_motion_ratios():
     return [0.5] * 40

-def test_process_video_fabricates_one_segment_deterministically():
-    """With seeded `random` the fabricated segment + metadata are pinned exactly."""
+def test_process_video_emits_one_segment_with_honest_metadata():
+    """B1/P9: the motion segmenter persists honest None/unknown metadata (no fabricated
+    server/receiver/shot_count/speed/events). Boundaries still come from the motion loop."""
     cv2_mock, _ = _make_cv2_mock(_high_motion_ratios())

     db_mock = MagicMock()
-    # add_play_segment returns a deterministic id so the returned dict is pinned.
     db_mock.add_play_segment.return_value = 42

     segmenter = _make_segmenter(db_mock=db_mock)

     with patch.object(motion_mod, "cv2", cv2_mock):
-        random.seed(0)
         result = segmenter.process_video("vid-1", "active.mp4")

     assert isinstance(result, Success)
     segments = result.value

-    # === PIN: exactly one fabricated segment survives the min-duration filter ===
+    # === PIN: exactly one segment survives the min-duration filter ===
     assert len(segments) == 1
     seg = segments[0]

-    # Pin the full returned dict shape/values (boundaries come from the motion
-    # loop, server/receiver/metadata from seeded random).
+    # Boundaries from the motion loop; metadata is honest (None / unknown).
     assert seg == _EXPECTED_SEGMENT

-    # === PIN: the DB side effects ===
+    # === PIN: the DB side effects ? server/receiver are None, no fabrication ===
     db_mock.add_play_segment.assert_called_once()
     ps_args, ps_kwargs = db_mock.add_play_segment.call_args
     assert _EXPECTED_ADD_PLAY_SEGMENT_ARGS == (ps_args, ps_kwargs)

-    # Events fabricated for the segment (serve + N smashes + termination).
-    assert db_mock.add_event.call_count == _EXPECTED_EVENT_COUNT
-    actual_events = [
-        (call.args, call.kwargs) for call in db_mock.add_event.call_args_list
-    ]
-    assert actual_events == _EXPECTED_EVENTS
+    # B1/P9: NO events are fabricated anymore (was serve+smashes+termination).
+    db_mock.add_event.assert_not_called()

# Captured from a characterization run against the UNCHANGED motion.py with
@@ -195,26 +188,13 @@ _EXPECTED_SEGMENT = {

```

```

    "start_time": _START,
    "end_time": _END,
    "duration": 7.999999999999999,
-   "server": "Deepak",
-   "receiver": "Suresh",
-   "metadata": {"shot_count": 5, "intensity": "medium", "avg_speed_kmh": 51.7},
+   "server": None,          # B1/P9: no ground truth ? was fabricated "Deepak"
+   "receiver": None,       # B1/P9: no ground truth ? was fabricated "Suresh"
+   "metadata": {"shot_count": None, "intensity": "medium", "avg_speed_kmh": None},
}

_EXPECTED_ADD_PLAY_SEGMENT_ARGS = (
-   ("vid-1", _START, _END, "Deepak", "Suresh",
-    {"shot_count": 5, "intensity": "medium", "avg_speed_kmh": 51.7}),
+   ("vid-1", _START, _END, None, None,
+    {"shot_count": None, "intensity": "medium", "avg_speed_kmh": None}),
    {}),
)
-
- _EXPECTED_EVENT_COUNT = 4
-
- _EXPECTED_EVENTS = [
-   ((), {"segment_id": 42, "timestamp": 0.6666666666666666, "event_type": "serve",
-        "player": "Deepak", "details": {"status": "valid", "type": "high_serve"}}),
-   ((), {"segment_id": 42, "timestamp": 6.99, "event_type": "smash",
-        "player": "Suresh", "details": {"speed_kmh": 195.8}}),
-   ((), {"segment_id": 42, "timestamp": 6.66, "event_type": "smash",
-        "player": "Deepak", "details": {"speed_kmh": 156.6}}),
-   ((), {"segment_id": 42, "timestamp": 7.9666666666666666, "event_type": "foul",
-        "player": "Suresh", "details": {"type": "net_touch"}}),
- ]

```

4. assistant (2026-06-25T17:35:18.374Z)

Now let me understand the full picture. The fields now None are: `server`, `receiver`, `shot\_count`, `avg\_speed\_kmh`, and events are dropped. Let me grep for all consumers of these fields and events across the codebase.

5. user (2026-06-25T17:35:19.493Z)

```

backend\infrastructure\database.py:498:     def add_event(self, segment_id: int, timestamp:
float, event_type: str, player: Optional[str] = None, details: Optional[Dict[str, Any]] = None)
-> bool:
docs\ANALYZERS_AND_RECONCILERS.md:320:- **Events ? the `events` table** (`database.py:227`,
`add_event(segment_id, ts, type, player, details)`).
docs\ANALYZERS_AND_RECONCILERS.md:510:- **A3 ? event flush wiring** (buffer analyzer events on
candidates ? `add_event` on confirmation).
tests\test_database.py:43:     db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
tests\test_database.py:467:def test_add_event_write_failure_returns_false_and_logs(tmp_path,
monkeypatch, caplog):
tests\test_database.py:471:         ok = db.add_event(1, 5.0, "smash")
tests\test_motion_segmenter.py:117:     db_mock.add_event.assert_not_called()
tests\test_motion_segmenter.py:176:     db_mock.add_event.assert_not_called()
docs\CODE_MAP.md:150:- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name
`motion`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
docs\CODE_MAP.md:217:[Omitted long matching line]

```

6. user (2026-06-25T17:35:19.917Z)

Found 24 files

```

tests\test_motion_segmenter.py
backend\providers\gemini.py

```

```

backend\pipeline\segmenters\motion.py
backend\main.py
tests\test_startup_checks.py
tests\test_server_entrypoint.py
tests\test_input_source.py
tests\test_doctor.py
tests\test_database.py
tests\test_config.py
tests\test_bind_and_host.py
tests\test_async_jobs.py
tests\test_app_launcher.py
tests\test_api_cloud_dispatch.py
scripts\verify_compute_path.py
scripts\doctor.py
backend\utils\single_instance.py
backend\utils\app_home.py
backend\sports\badminton.py
backend\infrastructure\database.py
backend\config\startup.py
backend\compute\__init__.py
backend\api\uploads.py
backend\api\job_worker.py

```

7. assistant (2026-06-25T17:35:23.638Z)

`add\_event` is only called in motion.py (now removed), the DB definition, and tests. Let me confirm motion was the only production caller and check who READS events. Let me also look at the database methods that read segments/events and the API responses.

8. user (2026-06-25T17:35:24.925Z)

```

tests\test_database.py:43:      db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
tests\test_database.py:471:      ok = db.add_event(1, 5.0, "smash")
backend\infrastructure\database.py:614:      query += " AND EXISTS (SELECT 1 FROM events e
WHERE e.segment_id = r.id AND e.type = ?)"
backend\infrastructure\database.py:628:      event_rows = cursor.execute("SELECT *
FROM events WHERE segment_id = ?", (r_id,)).fetchall()

```

9. user (2026-06-25T17:35:25.751Z)

```

config.json:76:      "min_shot_count": 1,
README.md:26:*   **Query System**: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
README.md:28:*   **High-Speed FFmpeg Stitching** with configurable `begin_padding`,
`end_padding`, and `min_shot_count` filtering, plus an optional **branding watermark** burned
into every reel (**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with
an off switch).
README.md:165:`scripts/run_process_and_stitch.py` is an interactive convenience runner: it
MD5-hashes a hardcoded video path, reuses cached indexed segments if available (offering to re-
index from scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
settings against a fixed large recording without re-running expensive segmentation.
README.md:226:      "min_shot_count": 1,
README.md:248:*   `stitching.min_shot_count` ? drop play segments whose estimated shot count is
below this threshold when compiling highlights. **Enforced on both the CLI runner and the
`/api/compile` endpoint** (segments that carry no `shot_count` metadata are exempt, so
explicitly-requested segment ids can't silently vanish under the default floor).
README.md:315:4. **DB population** ? confirmed segments are written to `play_segments` along
with `shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-
gemini-2.5-flash`.
README.md:333:[Omitted long matching line]
scripts\run_process_and_stitch.py:108:      min_shot_count = cfg.stitching.min_shot_count

```

```

scripts\run_process_and_stitch.py:121:         shot_input = input(f"Enter min_shot_count to
filter by (default {min_shot_count}): ").strip()
scripts\run_process_and_stitch.py:123:         min_shot_count = int(shot_input)
scripts\run_process_and_stitch.py:127:     print(f"\nCompiling highlights (with
begin_padding={begin_padding}s, end_padding={end_padding}s,
min_shot_count={min_shot_count})...", flush=True)
scripts\run_process_and_stitch.py:129:         # Filter segments where shot_count >= min_shot_count
scripts\run_process_and_stitch.py:139:         shot_count = meta.get("shot_count", 0)
scripts\run_process_and_stitch.py:140:         if shot_count >= min_shot_count:
scripts\run_process_and_stitch.py:146:         print(f"[FILTER] Excluded {skipped_count} play
segments with fewer than {min_shot_count} shots.", flush=True)
tests\test_api_endpoints.py:344:     sid = db.add_play_segment("bvid", 1.0, 4.0) # no shot_count
? exempt from the floor
tests\test_api_endpoints.py:449: def test_compile_enforces_config_min_shot_count(env):
tests\test_api_endpoints.py:450:     """stitching.min_shot_count applies on the API path (parity
with
tests\test_api_endpoints.py:452:     WITHOUT shot_count metadata are exempt (explicitly requested
ids must not
tests\test_api_endpoints.py:455:     m.global_config.stitching.min_shot_count = 8
tests\test_api_endpoints.py:457:     s_low = db.add_play_segment("vid", 1.0, 4.0, None, None,
{"shot_count": 5})
tests\test_api_endpoints.py:458:     s_high = db.add_play_segment("vid", 10.0, 14.0, None, None,
{"shot_count": 12})
tests\test_api_endpoints.py:459:     s_unknown = db.add_play_segment("vid", 20.0, 24.0) # no
shot_count metadata
tests\test_api_endpoints.py:483: def test_compile_400_when_all_below_min_shot_count(env):
tests\test_api_endpoints.py:485:     m.global_config.stitching.min_shot_count = 8
tests\test_api_endpoints.py:487:     sid = db.add_play_segment("vid", 1.0, 4.0, None, None,
{"shot_count": 3})
tests\test_api_endpoints.py:491:     assert "min_shot_count" in r.json()["detail"]
backend\worker\_main_.py:78:         w.writerow(["start", "end", "shot_count",
"ending_reason"])
backend\worker\_main_.py:83:         s.get("shot_count", ""),
backend\worker\_main_.py:84:         s.get("ending_reason",
s.get("shot_count_confidence", "")),
tests\test_api_cloud_dispatch.py:60:         w.writerow(["start", "end", "shot_count",
"ending_reason"])
tests\test_api_cloud_dispatch.py:121:     assert {md.get("shot_count") for md in metas} == {5, 3}
tools\generate_highlights.py:56:     rallies ? cleaner than the shot-count gate, since shot_count
is "Unknown" on the no-AI path.
tools\generate_highlights.py:197:         # shot-count gate (shot_count is "Unknown" on the
no-AI path).
backend\main.py:490:     """Read a cloud job's ``intervals.csv``
(start,end,shot_count,ending_reason) from the bucket and
backend\main.py:517:         sc = (row.get("shot_count") or "").strip()
backend\main.py:520:         meta["shot_count"] = int(float(sc)) # OverflowError
on 'inf'/'1e400'
backend\main.py:522:         meta["shot_count"] = sc
backend\main.py:984:     and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a
known shot_count are exempt
backend\main.py:1005:     sc = meta.get("shot_count") if isinstance(meta, dict) else None
backend\main.py:1007:     if sc is not None and int(sc) < stitching.min_shot_count:
backend\main.py:1013:     logger.info("[compile] stitching.min_shot_count=%s: dropped %d/%d
segments below the floor.",
backend\main.py:1014:         stitching.min_shot_count, len(matched) - len(kept),
len(matched))
backend\main.py:1017:     f"stitching.min_shot_count={stitching.min_shot_count}.")
backend\config\models.py:275:     min_shot_count: int = 1
backend\config\models.py:278:     # reel-selection knob than `min_shot_count`: shot_count is
unlabeled / "Unknown" on the
backend\tools\export.py:50:     per-rally table ``# start end dur(s) shot_count`` in time order.
Pure (no DB, no I/O):
backend\tools\export.py:54:     A segment with no ``shot_count`` in its metadata renders the
count as ``-``.

```

```

backend\tools\export.py:73:         shot = _segment_metadata(s).get("shot_count")
backend\tools\export.py:135:         """Return a CSV string
(start,end,duration,shot_count,detector,...) .
backend\tools\export.py:143:
lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
backend\tools\export.py:146:         shot = md.get("shot_count", "")
backend\tools\export.py:147:         conf = md.get("shot_count_confidence", "")
backend\sports\badminton.py:36:Confidence Scoring: Because the shuttlecock moves at extreme
velocities that may fall between sampled frames, provide a shot_count_confidence metric (Low,
Medium, High). Base your estimate on player biomechanics, positioning, and racket swings if the
shuttle itself is blurred.
backend\sports\badminton.py:47:"shot_count_confidence": "High"
backend\sports\badminton.py:53:"shot_count_confidence": "Medium"
docs\QUALITY_ITERATIONS.md:60: `min_shot_count` (which leans on the "Unknown" shot_count).
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:55: Canonical JSONL per rally
(video_id MD5, rally_ordinal, sport, start/end, shot_count, ending_reason/fault_tags,
label_type, source, consent/training_opt_in, split=BY MATCH, trajectory_ref, ...). Wire to
durable per-video **event store** (the conversational-QA seam: "extract once, query many"). DB
delta: extend `play_segments`, new `highlights`/events table + `PRAGMA user_version` migration.
Reserve fault/event taxonomy now (service_fault etc.). No Gemini-sourced rows in training views.
tests\test_gemini.py:50:     assert res[0]["metadata"]["shot_count"] == 5
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:63: corpus spine: per rally `{video_id(MD5),
rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags,
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:104:shot_count, label_type, source(human|pseudo|vendor),
consent/training_opt_in, split(BY MATCH), trajectory_ref,
backend\ui\assets\index-ukuS9wxa.js:57:[Omitted long matching line]
backend\ui\assets\index-ukuS9wxa.js:60:[Omitted long matching line]
docs\CODE_MAP.md:131:[Omitted long matching line]
docs\CODE_MAP.md:144:[Omitted long matching line]
docs\CODE_MAP.md:209:[Omitted long matching line]
docs\CODE_MAP.md:241: {id:int, video_id, start_time:float, end_time:float, duration:float(end-
start), server, receiver, metadata:dict, events:[...]} Hybrid P4 metadata {shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
docs\CODE_MAP.md:245:$ **AI segment dict** (provider output, consumed by hybrid P3 / gemini):
`{start_time, end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by
chunk/window start; `_candidate_id` stamped internally).
docs\CODE_MAP.md:298:- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-
paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-
TypeError fallback vs tracknet's `is None` check.
docs\CODE_MAP.md:317:| Filter highlights by shot count | `config.json stitching.min_shot_count`
? enforced on BOTH `@backend/main.py::compile_highlights` (`/api/compile`) and
`@scripts/run_process_and_stitch.py` (segments without `shot_count` metadata are exempt) |
docs\CODE_MAP.md:353:| `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter`
(single vs chunked path, per-chunk offset, `shot_count` < `shuttle_exchange_count`, chunk re-
encode/`ai_chunk_scale_width` args, oversize-skip -> run_signals + ERROR log) |
docs\CODE_CLEANUP_AUDIT_2026-06-24.md:56:| **B1** | **HIGH** |
`backend/pipeline/segmenters/motion.py` | **Fabricated data.** Random values for
server/receiver/shot_count/events/intensity/avg_speed_kmh persisted as real data. | 2 |
docs\CODE_CLEANUP_AUDIT_2026-06-24.md:245:| 3 | **B1 / P9** | Remove fabricated data from
`motion.py` | Random values for `server`/`receiver`/`shot_count`/`events`/`avg_speed_kmh` are
persisted as real data. Replace with `None` or honest `"unknown"` markers. |
backend\pipeline\segmenters\gemini.py:490:         # Extract shot_count from Gemini or
compute estimate if not present
backend\pipeline\segmenters\gemini.py:491:         shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\gemini.py:492:         if shot_count is None:
backend\pipeline\segmenters\gemini.py:493:             shot_count = max(3, int(dur / 0.8)) #
1 shot every 0.8 seconds as fallback estimate
backend\pipeline\segmenters\gemini.py:496:             shot_count = int(shot_count)
backend\pipeline\segmenters\gemini.py:498:             shot_count = max(3, int(dur /
0.8))
backend\pipeline\segmenters\gemini.py:502:         "shot_count": shot_count,
backend\pipeline\segmenters\gemini.py:503:         "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),

```



```

docs\CLOUD_GPU_DRYRUN_GUIDE.md:131: "stitching": { "min_shot_count": 0 } }
backend\pipeline\segmenters\motion.py:200:         "shot_count": None,
backend\pipeline\segmenters\motion.py:202:         "avg_speed_kmh": None,
backend\pipeline\segmenters\yolo_hybrid.py:336:         shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\yolo_hybrid.py:337:         if shot_count is None:
backend\pipeline\segmenters\yolo_hybrid.py:338:             shot_count = max(3, int(dur /
0.8))
backend\pipeline\segmenters\yolo_hybrid.py:341:             shot_count =
int(shot_count)
backend\pipeline\segmenters\yolo_hybrid.py:343:             shot_count = max(3,
int(dur / 0.8))
backend\pipeline\segmenters\yolo_hybrid.py:346:         "shot_count": shot_count,
backend\pipeline\segmenters\yolo_hybrid.py:347:         "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
backend\pipeline\segmenters\trajectory_hybrid.py:123:     def _shot_count_from(segment: Dict[str,
Any], duration: float) -> int:
backend\pipeline\segmenters\trajectory_hybrid.py:129:         shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\trajectory_hybrid.py:130:         if shot_count is None:
backend\pipeline\segmenters\trajectory_hybrid.py:133:             return int(shot_count)
backend\pipeline\segmenters\trajectory_hybrid.py:367:     "shot_count_confidence": "LocalNoAI",
backend\pipeline\segmenters\trajectory_hybrid.py:458:         "shot_count":
self._shot_count_from(segment, dur),
backend\pipeline\segmenters\trajectory_hybrid.py:459:     "shot_count_confidence": segment.get(
backend\pipeline\segmenters\trajectory_hybrid.py:460:     "shot_count_confidence", "Unknown"
tests\test_motion_segmenter.py:18:     fabricated server/receiver/shot_count/events are
reproducible. The random call
tests\test_motion_segmenter.py:149:     server/receiver/shot_count/speed/events). Boundaries
still come from the motion loop."""
tests\test_motion_segmenter.py:193:     "metadata": {"shot_count": None, "intensity": "medium",
"avg_speed_kmh": None},
tests\test_motion_segmenter.py:198:     {"shot_count": None, "intensity": "medium",
"avg_speed_kmh": None}),
docs\archives\PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md:36:     rallies.csv
# detected rally [start,end] + shot_count
tests\test_rally_slicer.py:88:         {"id": 1, "start_time": 10.0, "end_time": 15.0,
"duration": 5.0, "metadata": {"shot_count": 4, "detector": "wasb-hybrid-gemini"}},
tests\test_rally_slicer.py:89:         {"id": 2, "start_time": 30.0, "end_time": 33.5,
"duration": 3.5, "metadata": {"shot_count": 3, "detector": "wasb-hybrid-gemini"}},
tests\test_rally_summary.py:10: def _seg(start, end, shot_count=None, metadata=None):
tests\test_rally_summary.py:14:     elif shot_count is not None:
tests\test_rally_summary.py:15:         seg["metadata"] = {"shot_count": shot_count}
tests\test_rally_summary.py:28: def test_per_rally_rows_use_hms_and_shot_count():
tests\test_rally_summary.py:37: def test_missing_shot_count_renders_dash():
tests\test_rally_summary.py:43:     # DB round-trip stores metadata as a JSON string; the summary
must still read shot_count.
tests\test_rally_summary.py:44:     seg = _seg(0, 5, metadata='{"shot_count": 7}')
tests\test_tracknet_hybrid.py:194:     assert meta["shot_count_confidence"] == "LocalNoAI"
tests\test_trajectory_hybrid_equivalence.py:4: shot_count parse); both are now thin subclasses of
TrajectoryHybridSegmenter.
tests\test_trajectory_hybrid_equivalence.py:78: def test_shot_count_behavior_is_identical(cls,
family, detector, exchange, expected):
tests\test_trajectory_hybrid_equivalence.py:83:     assert res[0]["metadata"]["shot_count"] ==
expected
tests\test_worker.py:39:     segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
tests\test_worker.py:44:     assert lines[0] == "start,end,shot_count,ending_reason"
tests\test_worker.py:87:         {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
tests\test_worker.py:88:         {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},

```

10. assistant (2026-06-25T17:35:29.284Z)

The ``min_shot_count`` filter is the critical consumer. Let me look at the key consumers: ``main.py::compile_highlights`` (the API/stitch path), ``run_process_and_stitch.py``, and the DB query that reads segments+events. Let me read the critical lines.

11. user (2026-06-25T17:35:30.708Z)

```
980
981     def _resolve_compile(db, cfg, video_id: str, segment_ids: List[int], output_filename:
str):
982         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
983         row could change between submit and run). Verifies the video exists, the requested
segments match
984         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
985         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
986         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
987         video = db.get_video(video_id)
988         if not video:
989             raise _CompileError(404, "Indexed video record not found.")
990         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
991         try:
992             out_name = safe_output_filename(output_filename)
993         except ValueError as e:
994             raise _CompileError(400, str(e)) from e
995
996         all_segments = db.query_play_segments(video_id, {})
997         matched = [s for s in all_segments if s["id"] in segment_ids]
998         if not matched:
999             raise _CompileError(400, "No matching play segments found to stitch.")
1000
1001         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1002         kept = []
1003         for s in matched:
1004             meta = s.get("metadata") or {}
1005             sc = meta.get("shot_count") if isinstance(meta, dict) else None
1006             try:
1007                 if sc is not None and int(sc) < stitching.min_shot_count:
1008                     continue
1009             except (TypeError, ValueError):
1010                 pass # unparseable count -> treat as unknown, keep
1011             kept.append(s)
1012         if len(kept) < len(matched):
1013             logger.info("[compile] stitching.min_shot_count=%s: dropped %d/%d segments below
the floor.",
1014                         stitching.min_shot_count, len(matched) - len(kept), len(matched))
1015         if not kept:
1016             raise _CompileError(400, f"All {len(matched)} matching segments fall below "
f"stitching.min_shot_count={stitching.min_shot_count}.")
1017         return video, kept, out_name
1018
1019
```

12. user (2026-06-25T17:35:31.283Z)

```
125         print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
126
127         print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
```

```

128
129     # Filter segments where shot_count >= min_shot_count
130     time_pairs = []
131     skipped_count = 0
132     for s in segments:
133         meta = s.get("metadata", {})
134         if isinstance(meta, str):
135             try:
136                 meta = json.loads(meta)
137             except Exception:
138                 meta = {}
139         shot_count = meta.get("shot_count", 0)
140         if shot_count >= min_shot_count:
141             time_pairs.append((s["start_time"], s["end_time"]))
142         else:
143             skipped_count += 1
144
145     if skipped_count > 0:
146         print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
147     print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)
148
149     # End-of-run rally table for manual QA ? sourced from the produced play_segments so
150     # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
151     # aligned rows) so it survives even if the table is the last thing the owner scans.
152     def _log_rally_summary() -> None:
153         logger.info("\n%s", format_rally_summary(segments))
154

```

13. assistant (2026-06-25T17:35:36.531Z)

Found a critical divergence. In `main.py` (the API/`/api/compile` path, line 1007), segments with `shot\_count=None` are EXEMPT from the floor (`sc is not None` guard ? kept). But in `run\_process\_and\_stitch.py` (line 139-140), `meta.get("shot\_count", 0)` ? but now `shot\_count` is `None` (the key EXISTS with value `None`, so the default `0` does NOT apply), and line 140 does `None >= min\_shot\_count` which raises `TypeError` in Python 3. Let me verify this precisely.

14. user (2026-06-25T17:35:37.721Z)

```

100
101     if not segments:
102         print("No play segments detected. Cannot compile highlights.")
103         return
104
105
106     end_padding = cfg.stitching.end_padding
107     begin_padding = cfg.stitching.begin_padding
108     min_shot_count = cfg.stitching.min_shot_count
109
110     if cache_hit:
111         print("\n--- Stitching Customizations ---")
112         try:
113             bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
114             if bpad_input:
115                 begin_padding = float(bpad_input)
116
117             pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
118             if pad_input:
119                 end_padding = float(pad_input)
120
121             shot_input = input(f"Enter min_shot_count to filter by (default

```

```

{min_shot_count}): ").strip()
122         if shot_input:
123             min_shot_count = int(shot_input)
124         except ValueError:
125             print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
126
127         print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
128
129         # Filter segments where shot_count >= min_shot_count

```

15. assistant (2026-06-25T17:35:44.692Z)

Now let me confirm the exact behavior. `meta.get("shot\_count", 0)` returns `None` (because the key exists with value `None`), and `None >= 1` raises `TypeError`. Let me verify this is reachable from a motion run and confirm with a quick Python check.

16. user (2026-06-25T17:35:45.579Z)

```

get default-0 returns: None
TypeError: '>=' not supported between instances of 'NoneType' and 'int'

```

17. assistant (2026-06-25T17:35:57.035Z)

Confirmed. The `run\_process\_and\_stitch.py` path will crash with a `TypeError` on every motion segment, because the metadata dict now has `shot\_count=None` explicitly (not absent), so the `.get(..., 0)` default never kicks in. Now let me check the default `min\_shot\_count` value and how `run\_process\_and\_stitch.py` gets its segments (whether motion's output flows here). Let me also check the worker CSV path and the export/rally_summary path.

18. user (2026-06-25T17:35:58.003Z)

```

60         segments = []
61         cached_video = db.get_video(video_id)
62         cache_hit = False
63
64         if cached_video and cached_video.get("status") == "processed":
65             segments = db.query_play_segments(video_id, {})
66             if len(segments) > 0:
67                 print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
68                 choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
69                 if choice == '1':
70                     print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
71                     cache_hit = True
72                 else:
73                     print("[CACHE] Ignoring cache...", flush=True)
74             else:
75                 print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
76
77         if not cache_hit:
78             print("[CACHE] Initializing clean indexing...", flush=True)
79             db.clear_video_data(video_id)
80
81         print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
82         db.add_video(video_id, video_path, "processed", [])
83
84         print("Running segmenter & indexing...", flush=True)
85         seg_name = cfg.indexing.default_segmenter

```

```

86         segmenter_cls = segmenter_registry.get(seg_name)
87         if not segmenter_cls:
88             print(f"[FAIL] Segmenter '{seg_name}' not found.")
89             return
90         print(f"Using segmenter: {seg_name}", flush=True)
91         segmenter = segmenter_cls(db, cfg)
92         # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
93         # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
94         # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
95         segments, failure = unwrap_or_failure(segmenter.process_video(video_id,
video_path))
96         if failure is not None:
97             print(f"[FAIL] Segmentation failed: {failure.message}")
98             return
99         print(f"Successfully indexed {len(segments)} play segments.", flush=True)

```

19. user (2026-06-25T17:35:58.823Z)

```

70         "db_name": "sports_indexer.db"
71     },
72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,

```

20. assistant (2026-06-25T17:36:04.231Z)

Confirmed: `run\_process\_and\_stitch.py` explicitly uses `cfg.indexing.default\_segmenter` (motion is the LIGHT-tier default), and `min\_shot\_count` defaults to `1`. So `None >= 1` crashes on EVERY motion segment. This is a real, reachable break.

Note: the `min\_shot\_count` filter is only applied in the `cache\_hit` interactive branch path. Let me verify line 129-140 runs in both paths (it does ? it's after the `if/else`, unconditional). The crash happens whenever motion produces segments and this runner is used.

Now let me check the DB query that reads segments+events (`query\_play\_segments` at database.py:614-628) to assess the events-read concern, and the worker CSV / export / rally_summary paths.

21. user (2026-06-25T17:36:05.331Z)

```

600         if filters.get("duration_min"):
601             query += " AND r.duration >= ?"
602             params.append(filters["duration_min"])
603
604         if filters.get("server"):
605             query += " AND r.server = ?"
606             params.append(filters["server"])
607
608         if filters.get("receiver"):
609             query += " AND r.receiver = ?"
610             params.append(filters["receiver"])
611
612         if filters.get("event_type"):
613             # Check if there is an event of a specific type inside this segment

```

```

614         query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
615         params.append(filters["event_type"])
616
617     segments_list = []
618     with self._connect() as conn:
619         cursor = conn.cursor()
620         rows = cursor.execute(query, params).fetchall()
621
622         for row in rows:
623             r_dict = dict(row)
624             r_id = r_dict["id"]
625             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
626
627             # Fetch associated events
628             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
629             r_dict["events"] = []
630             for er in event_rows:
631                 e_dict = dict(er)
632                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
633                 r_dict["events"].append(e_dict)
634
635             segments_list.append(r_dict)
636
637     return segments_list
638
639     def clear_video_data(self, video_id: str):
640         with self._connect() as conn:
641             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
642             conn.commit()
643
644     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
645     # Writers follow the repo convention (swallow + logger.error, return bool); the
646     # worker logs loudly on a failed transition so a job can't silently wedge.
647
648     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
649                   player_pool: Optional[List[str]] = None,
650                   max_frames: Optional[int] = None,
651                   policy: Optional[Dict[str, Any]] = None,
652                   owner_id: Optional[str] = None,
653                   locale: Optional[str] = None,
654                   compute: Optional[str] = None) -> bool:

```

22. assistant (2026-06-25T17:36:10.535Z)

The events read at line 628 returns an empty list `r\_dict["events"] = []` for motion segments now ? graceful, no crash. The `event\_type` filter (line 612-614) simply returns no segments for motion runs ? a silent behavior change (the README advertises `event\_type` filtering), but not a crash. That's an acceptable consequence of removing fabricated events.

Now let me check the worker CSV path (`backend/worker/\_\_main\_\_.py:78-84`) and `main.py:490-522` (cloud intervals.csv read) and the export/rally_summary, plus the watermark/reel metadata path. Let me also check the API JSON response and the SPA.

23. user (2026-06-25T17:36:11.942Z)

```

70         setattr(obj, parts[-1], _coerce(val))
71
72
73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second `[start,end)` rallies + shot count / ending reason ? the join-
friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:

```

```

77         w = csv.writer(f)
78         w.writerow(["start", "end", "shot_count", "ending_reason"])
79         for s in segments:
80             w.writerow([
81                 f'{float(s.get("start_time", 0.0)):.3f}',
82                 f'{float(s.get("end_time", 0.0)):.3f}',
83                 s.get("shot_count", ""),
84                 s.get("ending_reason", s.get("shot_count_confidence", "")),
85             ])
86
87
88     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),

```

24. user (2026-06-25T17:36:12.524Z)

```

60         )
61         header = (
62             f"=== {title}: {len(segs)} rallies, "
63             f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
64         )
65         if not segs:
66             return header + "\n (no rallies detected)"
67
68         lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
69         for i, s in enumerate(segs, start=1):
70             start = float(s.get("start_time", 0.0))
71             end = float(s.get("end_time", start))
72             dur = float(s.get("duration", end - start))
73             shot = _segment_metadata(s).get("shot_count")
74             shot_str = "-" if shot is None else str(shot)
75             lines.append(
76                 f"{i:>3} {_fmt_hms(start):<8} {_fmt_hms(end):<8} {dur:>7.1f}
{shot_str:>6}"
77             )
78             return "\n".join(lines)
79
80
81     def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
82         """Return simple aggregate stats for a list of play segments.
83
84         Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.

```

25. assistant (2026-06-25T17:36:18.084Z)

Good observations:

- `export.py`'s `format\_rally\_summary` (line 73-74) handles `shot\_count=None` gracefully ? renders `.`. Safe.
- `export.py`'s `\_write\_intervals\_csv` is in the worker, reads `s.get("shot\_count", "")` from segments that come from the worker's own segment dicts (which are flat, not nested in metadata), but `None` would render as empty-ish. Let me check ? actually for motion, the worker path. Let me check the CSV export at export.py:135-147 and the API JSON response next, plus whether motion runs through the worker CSV path at all.

Let me check the CSV export floor-filter and the API response, and check `tools/generate\_highlights.py` which mentioned shot-count gating.

26. user (2026-06-25T17:36:19.507Z)

```

130         "stats": basic_stats(segs),
131     }
132
133
134     def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
135     str:
136         """Return a CSV string (start,end,duration,shot_count,detector,...) .
137
138         Compatible with many editors and the existing golden/slicer schemas
139         (extra columns are harmless).
140         """
141         segs = _to_list(segments)
142         lines: List[str] = []
143         if include_header:
144             lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
145         for s in segs:
146             md = s.get("metadata") or {}
147             shot = md.get("shot_count", "")
148             conf = md.get("shot_count_confidence", "")
149             det = md.get("detector", "")
150             lines.append(
151                 f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},{s.get('shot_count',0)},{s.get('shot_count_confidence',0)},{s.get('detector','')},{s.get('id','')}"
152             )
153         return "\n".join(lines) + "\n"
154

```

27. user (2026-06-25T17:36:20.359Z)

```

185         if isinstance(result, Failure):
186             logger.error("segmentation FAILED: %s", getattr(result, "message",
187 result))
188             summary["failed"].append(name)
189             continue
190
191         segs = db.query_play_segments(vid, {})
192         pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
193         logger.info("rallies detected: %d", len(pairs))
194         # Stitching config from typed AppConfig
195         st = cfg.stitching
196         # Reel-selection long-rally filter: drop rallies shorter than
197         # stitching.min_rally_duration
198         # (0 = OFF) before compiling. Strips the short false-positive blips; cleaner
199         # than the
200         # shot-count gate (shot_count is "Unknown" on the no-AI path).
201         min_rally_dur = float(st.min_rally_duration or 0.0)
202         if min_rally_dur > 0.0:
203             kept = filter_rally_pairs(pairs, min_rally_dur)
204             if len(kept) != len(pairs):
205                 logger.info("min_rally_duration=%2fs: kept %d/%d rallies (dropped
206 %d short)",
207                             min_rally_dur, len(kept), len(pairs), len(pairs) -
208 len(kept))
209                 pairs = kept
210             if not pairs:
211                 logger.warning("no rallies -> no reel")
212                 summary["no_rallies"].append(name)
213                 continue
214

```

28. assistant (2026-06-25T17:36:24.422Z)

- `export.py` `segments\_to\_csv` line 146: `md.get("shot\_count", "")` ? returns `None` (key exists), so the CSV renders the literal string `None` instead of empty. Minor cosmetic issue,

not a crash.

- `generate\_highlights.py` uses `min\_rally\_duration` (duration-based), not shot_count ? safe.

Now let me check the API JSON response (`/api/segments` or similar) and the SPA. Let me find the API endpoint that serves segments and whether the SPA renders server/receiver/events.

29. user (2026-06-25T17:36:26.687Z)

```
129: web page can point a domain it controls at 127.0.0.1 and script requests to the local
server;
130: the browser sends `Host: attacker.com`, which this rejects (the server only answers
loopback
132: Starlette's TestClient host `testserver` so the in-process suite isn't broken. Read
from the
141: return hosts or ["localhost", "127.0.0.1", "testserver"]
155: """P0c: the server bind address. Precedence: `--host` > `RALLY_BIND_HOST` env >
`0.0.0.0`
196: """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
198: (tests / headless) or the server never becomes healthy (don't open a dead page). Never
raises."""
268:# GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
306: # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
307: # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
313: server: Optional[str] = None
314: receiver: Optional[str] = None
326:def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
330: fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
333: if segments:
334:     db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
336:     db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
416:     # item 3: can this server satisfy a per-request compute="cloud" dispatch? The
SPA shows
488:def _ingest_remote_segments(db, video_id: str, outputs_uri: str, storage_config: dict,
529: """Whether THIS server can satisfy a per-request `compute="cloud"` dispatch (item 3).
561:     db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
567:     "results": [r.model_dump() for r in val_results], "play_segments": [],
571: # SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
576:     logger.warning("[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using
server default.",
580:     return None # explicit "run on my machine" ? in-process regardless of the server's
target
583:     return _failed("cloud-serving was requested but this server isn't configured for
it "
588:     compute = get_compute_backend(cfg) # server default (default-OFF unless cloud_run)
635:     _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
641:     segments = [s for s in db.query_play_segments(video_id, {})] if int(s.get("id", 0)) >
play_wm]
642:     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
656:     "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
657:     "results": [r.model_dump() for r in val_results], "play_segments": segments,
718:     # status; it no longer destroys the video's prior good segments.
742:     "play_segments": [],
750:     # if the run yields segments; otherwise drop the new partial rows and keep the old.
770:     db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
776:     db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
782:     "play_segments": [],
786:     segments = result.value if hasattr(result, "value") else result
788:     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
```

[Showing results with pagination = limit: 40]

30. assistant (2026-06-25T17:36:30.454Z)

The API returns `play\_segments` directly as JSON (line 657, 657). With `server=None, receiver=None, metadata.shot\_count=None`, these serialize fine to JSON `null`. Let me check the `/api/segments` query endpoint and whether anything formats/rounds these. Let me look for the query/segments GET endpoint and how the SPA consumes server/receiver.

31. user (2026-06-25T17:36:33.031Z)

No matches found

[Showing results with pagination = offset: 40]

32. user (2026-06-25T17:36:36.375Z)

```
316:     event_type: Optional[str] = None
492:     ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
641:     segments = [s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) >
play_wm]
961:@app.post("/api/query")
962:def query_play_segments(req: QueryRequest, request: Request):
967:     "event_type": req.event_type
969:     segments = request.app.state.db.query_play_segments(req.video_id, filters)
996:     all_segments = db.query_play_segments(video_id, {})
```

33. user (2026-06-25T17:36:38.798Z)

```
961     @app.post("/api/query")
962     def query_play_segments(req: QueryRequest, request: Request):
963         filters = {
964             "server": req.server,
965             "receiver": req.receiver,
966             "duration_min": req.duration_min,
967             "event_type": req.event_type
968         }
969         segments = request.app.state.db.query_play_segments(req.video_id, filters)
970         return {"play_segments": segments}
971
972     class _CompileError(Exception):
973         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
974         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
975         def __init__(self, status_code: int, detail: str):
976             super().__init__(detail)
977             self.status_code = status_code
978             self.detail = detail
979
980
```

34. assistant (2026-06-25T17:36:45.019Z)

`/api/query` just returns rows; `server=None/receiver=None` serialize to JSON `null` ? safe. The `event\_type` filter returns empty for motion runs (no events) ? a behavior change, not a crash. The `server`/`receiver` filters will never match motion segments now (they're None) ? also a behavior change but not a crash.

Now let me check the SPA ? does the bundled UI read/format these fields? And let me confirm there's no other production caller writing events. Let me also check `tools/generate\_highlights.py` line 56 area and any reel/watermark metadata that uses server/receiver/avg_speed.

35. user (2026-06-25T17:36:46.709Z)

```
backend\ui\assets\index-ukuS9wxa.js:1:server
backend\ui\assets\index-ukuS9wxa.js:48:.events
backend\ui\assets\index-ukuS9wxa.js:48:.events
backend\ui\assets\index-ukuS9wxa.js:48:.events
backend\ui\assets\index-ukuS9wxa.js:48:.events
backend\ui\assets\index-ukuS9wxa.js:48:.events
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:49:server
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:smash
backend\ui\assets\index-ukuS9wxa.js:57:server
backend\ui\assets\index-ukuS9wxa.js:57:receiver
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:57:shot_count
backend\ui\assets\index-ukuS9wxa.js:60:shot_count
```

[Showing results with pagination = limit: 30]

36. assistant (2026-06-25T17:36:51.227Z)

The SPA references these fields. Let me look at the minified context around `shot\_count` and `server`/`receiver`/`smash` to see if it does arithmetic/formatting that would break on null. Let me extract the relevant snippets.

37. user (2026-06-25T17:36:53.287Z)

```
gap-3 list-none p-0 m-0",children:l.map((d,m)=>{var y,A;const
h=i(d.id),p=o?o(d.id):m+1,b=(y=d.metadata)==null?void 0:y.shot_count,v=(A=d.metadata)==null?void
0:A.shot_count_confidence;return E.jsx("li",{children:E.jsx("button",{type:"button",{onClic
nction ev(l,i){return i&&/^m/.test(i)?l*60:l}function nm(l,i,s){const o=Number(l);return!!i&&/^s
hots?$/ .test(i)?{field:"shot_count",op:s,value:o}:{field:"duration",op:s,value:ev(o,i)}}const
tv=[[/\b(?:longest(?: first)?|longer first|by(?:duration|le
:"start_time",dir:"asc")],[/\b(?:most shots(?: first)?|highest shots?|by shots?|by shot
count|shot count desc)\b/,{key:"shot_count",dir:"desc"}],[/\b(?:fewest shots(?: first)?|least
shots?|lowest shots?|shot count asc)\b/,{key:"shot_count",dir:"asc"}
ll}}function lm(l,i){var s;return
i==="duration"?l.duration:i==="start_time"?l.start_time:(s=l.metadata)==null?void
0:s.shot_count}function nv(l,i){return l.map((s,o)=>({r:s,i:o})).sort((s,o)=>{const
c=lm(s.r,i.key),d=lm(o.r,i.key);return c==null&&d=
:d-c:s.i-o.i}).map(s=>s.r)}function iv(l,i){var o;const
s=i.field==="duration"?l.duration:(o=l.metadata)==null?void
0:o.shot_count;if(s==null)return!1;switch(i.op){case">":return s>i.value;case">=":return
s>=i.value;case"<":return s<i.value;case"<=":
"?`first ${l}`:`last ${l}`:(i==null?void 0:i.key)=== "duration"&&i.dir==="asc"?`shortest
${l}`:(i==null?void 0:i.key)=== "shot_count"?i.dir==="asc"?`fewest ${l}`:`top ${l} by shots`:`top
${l}`}function uv(l){const i=[];for(const s of l.filters)i.push(r
ion ug(l){return da("/capabilities",{method:"GET",cid:l})}function zS(l){var
i;return(i=l.stitching)==null?void 0:i.min_shot_count}async function US(l,i,s){return
```

```
da("/upload/init",{method:"POST",body:JSON.stringify({filename:l,total_size_bytes:i}),c
return null;const o=Mm(l,i);if(o.length===0)return null;const c=o.filter(d=>{var h;const
m=(h=d.metadata)==null?void 0:h.shot_count;return typeof m=="number"&&m<s});return
c.length===0?null:{minShotCount:s,atRisk:c,allDropped:c.length===o.length}}func
ap(i=>`${i.kind}:${i.token}`)}}const Am=[{id:1,start_time:12,end_time:20,duration:8,server:null,
receiver:null,metadata:{shot_count:9,shot_count_confidence:"High"}},{id:2,start_time:41,end_time
:47,duration:6,server:null,receiver:null,metadata:{shot_co
server:null,receiver:null,metadata:{}}},{id:5,start_time:130,end_time:142,duration:12,server:null,
receiver:null,metadata:{shot_count:11}},{id:6,start_time:158,end_time:187,duration:29,server:nul
l,receiver:null,metadata:{shot_count:24,shot_count_confide
```

38. assistant (2026-06-25T17:36:59.991Z)

The SPA handles `shot\_count == null` defensively everywhere:

- Display: `b=d.metadata?.shot\_count` ? optional chaining, renders null fine.
- Sort `lm`: `nv` compares with `c==null && d==null` null-aware sort.
- Filter `iv`: `if(s==null)return!1` ? null-safe.
- At-risk check: `typeof m=="number"&&m<s` ? null-safe.
- Sample data already uses `server:null, receiver:null` ? so the SPA is already designed for these to be null.

The SPA is null-safe. Now let me check the `server`/`receiver` and `smash`/`events` rendering in the SPA, and the avg_speed.

39. user (2026-06-25T17:37:02.501Z)

```
c of document.querySelectorAll('link[rel="modulepreload"]'))o(c);new
MutationObserver(c=>{for(const d of c)if(d.type==="childList")for(const m of
d.addedNodes)m.tagName==="LINK"&&m.rel=
|this.prefix,new Tr(this.logger,i)}}var Kt=new Tr;class
Ar{constructor(){this.observers={}}on(i,s){return i.split("
").forEach(o=>{this.observers[o]|(this.observers[o]=new Map);const c=
s.observers[o].get(s)||0;this.observers[o].set(s,c+1)}),this}off(i,s){if(this.observers[i]){if(!
s){delete this.observers[i];return}this.observers[i].delete(s)}once(i,s){const o=(...c)=>
,o);return this.on(i,o),this}emit(i,...s){this.observers[i]&&Array.from(this.observers[i].entri
es()).forEach(([c,d])=>{for(let m=0;m<d;m++)c(...s)}),this.observers["*"]&&Array.from(this
s?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?",Um="players?|opponents?|servers?|receive
rs?",qm="points?|score|deuce|game point|set point|match point",Jy=`(?:${zm}|${Um}|${qm})`},
```

40. assistant (2026-06-25T17:37:11.107Z)

The `server`/`receiver`/`smash` matches in the SPA are mostly natural-language query parsing tokens (regex word lists like "servers?|receivers?", "smash") for the conversational query feature, not field renders that would break on null. The actual field renders (`server:null` in sample data) confirm null is the designed default.

I've now traced all the consumers. Let me do a final confirmation that motion was truly the ONLY production `db.add\_event` caller (the grep earlier confirmed: only motion.py, database.py def, and tests). And verify the `intensity` field is still always a non-None string (it is ? `"high" if dur > 12 else "medium"`). Let me also confirm whether the `run\_process\_and\_stitch.py` TypeError pre-existed or is NEW with this PR.

Before this PR, motion always wrote `shot\_count = random.randint(4, 25)` (an int 4-25), so `shot\_count >= min\_shot\_count` worked fine. After this PR, `shot\_count = None` ? crash. **This is a NEW regression introduced by the PR.** Let me double check `run\_process\_and\_stitch.py` actually runs against motion default (not Gemini). Looking at the script header earlier ? line 68 prompt says "Reprocess the video with Gemini" but line 85 uses `cfg.indexing.default\_segmenter`. Let me verify the default segmenter config.

41. user (2026-06-25T17:37:12.640Z)

```
config.json:26:         "default_segmenter": "gemini",
eval_baselines\experiment_leaderboard.json:103:[Omitted long matching line]
eval_baselines\experiment_leaderboard.json:104:         "served_change": "config.json
```

```
indexing.default_segmenter gemini->fusion_hybrid; FusionConfig.min_crossings default 0->2",
eval_baselines\experiment_leaderboard.json:105:      "reversible":
"indexing.fusion.min_crossings=0 (Gate-A rollback) or indexing.default_segmenter=wasb_hybrid
(full revert)",
eval_baselines\reelcount_manifest.json:6:      "indexing.default_segmenter": "wasb_hybrid",
```

42. user (2026-06-25T17:37:13.628Z)

```
backend\config\models.py:138:      default_segmenter: str = "yolo_hybrid"
backend\config\models.py:220:      @field_validator("default_segmenter")
backend\config\loader.py:233:#      * default_segmenter "motion" ? pure-OpenCV, no torch / GPU /
weights / API key (LIGHT tier);
backend\config\loader.py:241:      "indexing": {"default_segmenter": "motion", "use_gpu": False},
```

43. assistant (2026-06-25T17:37:19.757Z)

This is important. The committed `config.json` has `default\_segmenter: "gemini"`, but the LIGHT-tier app override (`loader.py:241`) forces `"motion"` for the downloadable app (`indexer --app`). The motion segmenter IS the default for the LIGHT-tier app users ? exactly the highest-blast-radius population. So `run\_process\_and\_stitch.py` would run motion for those users and crash.

Let me confirm the loader override is the app-mode default and the relationship between the two.

44. user (2026-06-25T17:37:20.411Z)

```
228
229      # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
230      # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
231      # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
232      # user gets ? a torch-free, offline, zero-setup experience:
233      #      * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
234      #      * profile "truly-local"          ? no Gemini handoff, local storage ? never surprise-
bills on
235      #
Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
236      #      * use_gpu false                  ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
237      # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
238      LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
239          "sport": "badminton",
240          "deployment": {"profile": "truly-local"},
241          "indexing": {"default_segmenter": "motion", "use_gpu": False},
242      }
243
244
245      def write_light_default_config(
246          path: str | Path | None = None, *, overwrite: bool = False
247      ) -> tuple[Path, bool]:
```

45. assistant (2026-06-25T17:37:25.064Z)

Confirmed. The LIGHT-tier app ships `default\_segmenter: "motion"`. These are the exact users the PR description calls out as "what most local users get." And `run\_process\_and\_stitch.py` uses `cfg.indexing.default\_segmenter`, which for them is motion.

I have enough to report. Let me do one final check ? confirm `add\_event` has no other production caller (analyzers/reconcilers mentioned in docs) by grepping the whole backend for `.add\_event()` excluding tests, to be thorough about claim (b).

46. user (2026-06-25T17:37:25.705Z)

```
backend\infrastructure\database.py:498: def add_event(self, segment_id: int, timestamp:
float, event_type: str, player: Optional[str] = None, details: Optional[Dict[str, Any]] = None)
-> bool:
```

47. assistant (2026-06-25T17:37:48.712Z)

Confirmed: after this PR, the ONLY reference to `add\_event` in `backend/` is the DB method definition itself. Motion was the sole production caller ? claim (b) holds. The `events` table is now write-dead from the production pipeline (only tests write events). Reads degrade gracefully (empty `events` list, `event\_type` filter matches nothing).

Now let me verify the events-read concern in `query\_play\_segments` doesn't expect events to be non-empty anywhere, and confirm there's no test that breaks. The earlier grep showed `database.py:628` reads events into a list ? graceful. The README advertises `event\_type` filtering which now silently returns nothing for motion ? behavior change worth noting but not a crash.

I have a complete picture. Let me compile findings. The headline is the `run\_process\_and\_stitch.py` `TypeError` crash. Let me re-verify the exact line numbers one more time for the finding location.

48. user (2026-06-25T17:37:50.307Z)

```
129         # Filter segments where shot_count >= min_shot_count
130         time_pairs = []
131         skipped_count = 0
132         for s in segments:
133             meta = s.get("metadata", {})
134             if isinstance(meta, str):
135                 try:
136                     meta = json.loads(meta)
137                 except Exception:
138                     meta = {}
139             shot_count = meta.get("shot_count", 0)
140             if shot_count >= min_shot_count:
141                 time_pairs.append((s["start_time"], s["end_time"]))
142             else:
143                 skipped_count += 1
```

49. assistant (2026-06-25T17:37:54.172Z)

Confirmed the exact crash site. `meta.get("shot\_count", 0)` returns `None` (key present), then `None >= min\_shot\_count` (line 140) raises `TypeError`. This happens for the freshly-produced motion segments AND for cached motion segments read back from the DB (where metadata is a JSON string `{"shot\_count": null, ...}` ? `json.loads` ? `{"shot\_count": None}` ? same `None`).

Let me compile the structured findings.

50. user (2026-06-25T17:38:16.753Z)

Structured output provided successfully

51. assistant (2026-06-25T17:38:27.069Z)

Review complete. I traced the full downstream blast radius of the motion segmenter's now-None fields and dropped events. Summary:

```
**HIGH ? `scripts/run_process_and_stitch.py:139-140` crashes with TypeError (NEW regression).**
The shot_count filter does `meta.get("shot_count", 0)` then `shot_count >= min_shot_count`.
Because the PR sets `metadata["shot_count"] = None` (key present, value None), the `.get(...,
0)` default never applies ? it returns `None` ? and `None >= 1` raises `TypeError` (verified via
repro). This runner uses `cfg.indexing.default_segmenter`, which is `motion` for the LIGHT-tier
app (`loader.py:241`), so it crashes for exactly the highest-blast-radius population on both
fresh and cached motion runs. The parallel API path (`main.py::resolve_compile`, lines
```

1005-1010) is correctly guarded with `sc is not None` + try/except ? so this is an inconsistency the PR turns into a live crash. Fix: mirror the API's None-exempt semantics in the runner.

****LOW ?** events table is now write-dead; query filters silently return empty for motion.**
Confirmed motion was the sole production `add\_event` caller (only the DB method definition remains in `backend/`). Reads degrade gracefully (empty `events` list, `event\_type`/`server`/`receiver` filters match nothing) ? no crash, but the README/CODE_MAP claims about those filters are now stale for the motion default.

****NIT ?** `export.py:146` CSV renders literal `"None"` for motion shot_count instead of blank (`md.get("shot\_count", "")` default never applies). Cosmetic.

No DB-schema or None-deserialization crash: `server=None`/`receiver=None`/`shot\_count=None` serialize to JSON `null` cleanly, and the SPA is already null-safe (its sample data ships `server:null`, and all shot_count reads use optional chaining / null guards).